NVIDIA CVDP Agent: A Codex CLI Wrapper for General-Purpose Verilog Design Automation

Samarth Bangalore Sudharshan · Rijul Rajendra Wankhade

Mini Project 2

1. Motivation for the Problem

RTL design bugs a pipeline register one stage too deep, an accumulation counter whose enable never fires, a PRBS polynomial with a single wrong tap bit are expensive to debug manually. Static linting cannot fix them because the fix requires understanding design intent from a natural-language specification. LLMs have strong code-generation capability, but one-shot generation fails on complex hardware: the model produces plausible RTL that compiles but fails simulation, with no path to correct it.

An agent-based approach closes this gap. By wrapping an LLM in an iterative loop that feeds real simulation output back as context, the agent can diagnose failures and converge on correct RTL mimicking the human debug cycle at inference speed. The NVIDIA ICLAD25 CVDP benchmark evaluates this capability across 30 diverse Verilog problems graded by an official cocotb/pytest harness.

2. Problem Definition

The NVIDIA CVDP (Circuit Verification and Design Problems) benchmark consists of 30 Verilog problems spanning three task types: Spec2RTL (generate a module from a natural-language specification), RTL Modification (fix or complete a partial implementation), and DV Debugging/Bugfix (diagnose and repair a functionally incorrect module). Problems are graded by difficulty: 1 easy, 18 medium, 11 hard.

Inputs

- JSONL dataset: each entry contains the problem id, categories (task type + difficulty), a natural-language prompt, context files (initial RTL and local testbench), and a harness field containing official cocotb/pytest test specifications.
- AGENTS.md: a persistent instruction file (analogous to CLAUDE.md) injected into every problem working directory, encoding 8 RTL design rules derived from failure analysis.

Outputs

- Fixed or generated Verilog files written to rtl/ inside each problem's working directory.
- results_retry.json: Phase A summary (pass/fail/unverified per problem).
- raw_result.json / report.txt: official Phase B grading results (result: 0 = PASS, 1 = FAIL).

Objectives

- Maximize the fraction of problems graded PASS by the official NVIDIA cocotb/pytest harness.
- Research question: how effectively can an LLM agent solve RTL design problems when given maximal context harness specs, accumulated error logs, and explicit RTL design rules? A secondary question: does iterative feedback-based repair outperform one-shot generation? Answer to both: yes — 50% → 83%, with the largest gains on hard problems (27% → 82%).

3. System Overview

The pipeline runs in two phases. Phase A generates RTL on the Windows host using Codex CLI as the agent. Phase B grades the generated RTL using the official NVIDIA Docker harness running in WSL (Ubuntu 22.04).

Phase A — RTL Generation (Windows)	Phase B — Official Grading (WSL + Docker)
Read JSONL → setup_workdir() (extract rtl/, verif/, harness/src/, write prompt.txt + AGENTS.md)	run_benchmark.py launches harness
codex exec reads AGENTS.md,	Container 1 (cvdp-relay-agent):

fixes rtl/, runs iverilog internally	copies pre-built RTL → /code/rtl/
run_iverilog() — independent iverilog + vvp validation	Container 2 (ghcr.io/hdl/sim/osvb): iverilog + cocotb/pytest
Pass → done | Fail → error_log.txt → escalate prompt → next attempt	result: 0 = PASS result: 1 = FAIL result: 2 = ERROR
results_retry.json	raw_result.json + report.txt

Why a relay agent? The NVIDIA harness always launches an agent container, but Codex CLI requires Node.js and an interactive terminal it cannot run inside Docker. The relay container (cvdp-relay-agent) bridges this: it copies RTL that Phase A already generated on the host into the harness mount point, then exits. Phase B's grader container then runs the official cocotb tests against the copied RTL.

4. Agent Architecture

Our agent follows the ReAct pattern: Reason → Act → Observe → Repeat. The four key components are:

Escalating Prompts

The planner decides what instruction to give Codex on each attempt. Rather than a fixed prompt, it escalates across up to 6 attempts:

Attempt	Focus
1	Read AGENTS.md, fix rtl/
2	Read error_log.txt, understand failures
3	Think before changing anything
4	Fix only what fails, preserve what works
5	Trace every signal against spec
6	Verify every testbench assertion

Executor

The executor calls codex exec --dangerously-bypass-approvals-and-sandbox --skip-git-repo-check as a subprocess with the problem's working directory as cwd. Codex reads AGENTS.md automatically, then reads prompt.txt, the context RTL files, and the harness spec files. It writes fixes directly to rtl/ and runs iverilog internally. Model: gpt-5.4 with reasoning_effort=high (ASU OpenAI account).

iverilog + vvp Validation

After each Codex call, run_iverilog() independently compiles and simulates the RTL. It scans stdout for PASS/FAIL markers and checks vvp exit code. For problems with placeholder testbenches (15/30), it checks compile success only and marks the result as 'unverified'. iverilog + vvp output is appended to error_log.txt in the workdir.

error_log.txt Accumulation

The workdir is never reset between retries. Each failed attempt appends its iverilog and vvp output to error_log.txt, giving Codex a growing history of what failed and why. For unverified problems, a self-review mechanism runs after compile success: a dedicated Codex call reads harness/src/ + rtl/, traces every assertion, and writes REVIEW VERDICT: PASS or FAIL. A FAIL verdict feeds into error_log.txt and continues the retry loop.

5. Tools & Frameworks

Tool / Framework	Version	Role
Codex CLI (OpenAI)	0.118.0	LLM agent engine — reads files, writes RTL, runs shell commands
GPT-5.4	reasoning: high	Underlying model; ASU OpenAI account; not called directly
iverilog + vvp	12.0	Local Verilog compilation and simulation (Phase A validation)
cocotb	1.9.0 (pinned)	Python-based hardware verification co-simulation (Phase B)
pytest	latest	Test runner for cocotb harness inside Docker container
ghcr.io/hdl/sim/osvb	latest	Official NVIDIA grader container (iverilog + cocotb)
Docker Desktop + WSL2	Ubuntu 22.04	Phase B execution environment; required for os.sync()
Python 3.12	3.12.0	agent.py orchestrator, run_benchmark.py harness runner
AGENTS.md	v3 (8 rules)	Persistent Codex instruction file — injected per workdir

6. Context Engineering

Context engineering is the single most impactful lever in our system. We provide Codex with four structured information sources:

1. AGENTS.md

An 8-rule RTL design guide derived from failure analysis across all 30 problems. Rules cover: latency and pipeline depth counting, preventing undriven outputs (high-Z), tracing accumulation logic, verifying every asserted signal (not just primary outputs), exact algorithm constants (PRBS polynomials, encryption keys), parameterized boundary cases (index overflow when A == B), state preservation on no-match inputs, and FSM output duration (full state vs transition cycle). Codex reads this file automatically from its working directory on every attempt.

2. harness/src/ — Cocotb Spec Files

The JSONL harness field contains the official cocotb/pytest test files. We write all .py files (excluding test_runner.py, which reads Docker env vars) into workdir/harness/src/. Codex reads these as its primary specification they contain exact signal names, expected output values, latency constants, and parameter sweeps that the natural-language prompt alone underspecifies. This is the most information-dense context source and had the largest per-change impact on hard problems.

3. error_log.txt

After each failed attempt, iverilog and vvp output is appended to error_log.txt in the workdir. The file grows across retries. Codex is explicitly told to read it in attempts 2–6. Context size is managed naturally: iverilog errors are concise (line numbers and messages), and vvp simulation output is bounded by testbench $display statements. No summarization or truncation is needed for 6-attempt runs.

4. prompt.txt

The JSONL prompt field is written to prompt.txt. This provides the human-readable design intent. The JSONL system_message (written for the original ICLAD Linux harness, using sed/awk/patch) is intentionally excluded it is incompatible with Codex CLI on Windows and would confuse the agent.

7. Improvements from Phase 2

Phase 2 established the baseline pipeline (one-shot Codex, local iverilog, no harness spec, no AGENTS.md). Phase 3 introduced five targeted improvements, each measured against the previous configuration:

Improvement	Delta	Key Mechanism
Harness spec visibility (harness/src/ in workdir)	+1 problem (50%→53%)	Cocotb spec files give Codex signal-level ground truth
Retry mode (6 attempts + error_log)	+2 problems (53%→60%)	Iterative feedback loop; hard problems benefit most
cocotb 1.9.0 version pin (osvb image fix)	+2 problems (60%→67%)	3 problems had correct RTL but harness crashed on cocotb 2.0
AGENTS.md v2 (5 RTL rules + extraction checklist)	+4 problems (67%→80%)	Explicit rules for latency, high-Z, accumulation logic
AGENTS.md v3 (8 rules) + harness allowlist fix	+1 problem (80%→83%)	elevator_control.py was silently excluded; one-line fix → PASS

8. Results

Best result: 25 / 30 problems (83.3%), 30 / 35 tests (85.7%)

Configuration	Problems	Tests
Baseline (one-shot, no harness spec)	15/30 50.0%	20/35 57.1%
+ Harness spec (one-shot)	16/30 53.3%	21/35 60.0%
+ Retry mode	18/30 60.0%	23/35 65.7%
+ cocotb 1.9.0 pin	20/30 66.7%	25/35 71.4%
+ AGENTS.md v2 (5 rules)	24/30 80.0%	29/35 82.9%
+ AGENTS.md v3 + allowlist fix	25/30 83.3%	30/35 85.7%

Difficulty	Baseline	Final	Delta
Easy (1)	0/1 (0%)	1/1 (100%)	+100%
Medium (18)	12/18 (66.7%)	15/18 (83.3%)	+16.6%
Hard (11)	3/11 (27.3%)	9/11 (81.8%)	+54.5%

Generalization to Extended Dataset

The agent was evaluated on the full NVIDIA CVDP HuggingFace dataset (92 problems, cvdp_agentic_no_commercial), which has zero ID overlap with the 30 hackathon problems. Result: 65/92 problems passed (70.65%), 68/97 tests (70.10%). The lower overall rate relative to the hackathon set reflects the harder difficulty distribution (20 hard vs 11) and no dataset-specific tuning AGENTS.md rules were derived solely from the 30 hackathon failures.

Analysis

Why it improved: Retry mode is the largest single driver hard problems went from 27% (one-shot) to 82% (retry). The feedback loop lets Codex build on its own previous changes rather than starting fresh. AGENTS.md rules directly addressed the most common failure class: latency off-by-one (register depth miscounts) and accumulation logic bugs. Harness spec visibility had the highest per-problem impact on hard problems, where natural-language prompts underspecify timing constraints.

Where it failed: The 5 remaining failures all require real cocotb simulation to detect latency off-by-one (compass_breeze_obsidian: 22 vs 21 cycles), high-Z boundary conditions (breeze_velvet_violet: ARRAY_SIZE=6), structural RTL logic errors (lagoon_dragon_diamond: sequence detection never fires), and an exact bit-pattern requirement (thunder_diamond_horizon: PRBS polynomial all 73 parameter combinations fail). Prompt engineering cannot reliably fix any of these.

9. Challenges, Limitations, and Future Improvements

Key Challenges

- Phase A / Phase B oracle mismatch: 15/30 problems have only a placeholder local testbench. Phase A has no simulation feedback for these only compile success. Phase B is the only authoritative oracle, creating a closed-loop limitation.
- The self-review mechanism fires correctly and Codex does run real simulations it writes Verilog testbenches using mktemp, compiles with iverilog, and iterates on failures. However, its testbench counted 6 cycles for the smallest-key BST delete while the official harness measured 7, and its boundary-case coverage missed the ARRAY_SIZE=6 high-Z condition entirely. The failure is a cycle-counting protocol mismatch and incomplete test coverage, not hallucination. Only the official cocotb harness is a reliable oracle for these failure classes.
- Infrastructure bugs consumed significant Phase 3 effort: the cocotb 2.0/1.9 version mismatch, Docker PATH in non-interactive WSL subprocesses, argparse action='append' bug, and harness allowlist (elevator_control.py exclusion) each required diagnosis before any RTL-level work could proceed.

Limitations

- Scalability: 6 Codex calls × gpt-5.4 with high reasoning effort × 30 problems is expensive in tokens and wall time. Not practical without institutional API access.
- Generalization: AGENTS.md rules were derived from this dataset's 30 problems. Rules may not transfer to other RTL benchmarks with different failure distributions.
- Runtime: Phase A takes approximately 3–4 hours for all 30 problems in retry mode. Phase B adds 1–2 hours for Docker harness runs.

Future Improvements

- Real simulation loop for unverified problems: a lightweight Docker wrapper that runs cocotb without the full NVIDIA harness overhead would close the Phase A feedback loop for 15/30 problems the single largest remaining bottleneck.
- Multi-LLM architecture: separate sub-agents for RTL repair vs generation (e.g. a fast model for compile-error fixes, a high-reasoning model for spec-to-RTL), coordinated by an orchestrator.
- Adaptive timeout: heuristic-based Codex timeout scaled to problem complexity (number of signals, harness assertion count) rather than a fixed 20-minute cap for all problems.
- AGENTS.md smoke-test section: add a lightweight self-contained smoke test stub per category so Codex can run a minimal local check before committing RTL changes.
- Dynamic AGENTS.md: generate rules empirically per problem category by analyzing harness spec patterns, rather than maintaining a fixed global ruleset.

10. AI Usage and Insights

Role of AI in the System

Codex CLI (gpt-5.4) is the executor — it reads Verilog files, reasons about design intent from the harness spec and AGENTS.md rules, writes fixes to rtl/, and runs iverilog internally. Our Python orchestrator is a thin wrapper: it sets up the environment, invokes Codex as a subprocess, and parses the results. The LLM does all RTL reasoning.

Where AI Works Well

- RTL repair with clear error messages: when iverilog produces a specific compile error (undefined signal, port mismatch), Codex reliably fixes it in the next attempt.
- Spec-to-RTL when the spec is precise: problems where the cocotb test file specifies exact signal drives and expected values (cid003, cid016) had a 100% final pass rate.
- Incremental improvement: retry mode significantly outperforms one-shot on all difficulty levels, confirming that LLMs benefit from structured feedback loops.
- Generalization: rules and architecture derived from 30 hackathon problems transferred to a 92-problem HuggingFace dataset with zero overlap, achieving 70.65% without any dataset-specific tuning.

Where AI Fails

- Self-review oracle mismatch: Codex does run real simulations in self-review mode, but its self-written testbenches use different cycle-counting conventions than the official cocotb harness and miss parameterized boundary cases. For example, Codex measured 6 cycles for a BST delete where the harness measured 7. Even an explicit AGENTS.md rule to count pipeline stages cannot fix this — the failure is undetectable without the official oracle. 3 of the 5 remaining failures fall into this class.
- Exact bit patterns: PRBS polynomials, encryption S-box constants — any value that must match a specific hardware standard exactly cannot be inferred from natural-language prompts alone, regardless of retry depth.

Experience with AI in Development

Claude Code was used throughout development: for writing and debugging agent.py, diagnosing Phase B infrastructure failures, analyzing harness reports to identify RTL bugs, and drafting AGENTS.md rules. AI accelerated all phases of development significantly — infrastructure bugs that would have taken hours of manual log analysis were diagnosed in minutes by providing harness output as context.

The key limitation: AI is most valuable as a reasoning accelerator when the human provides the right observation (the failing log, the specific error). When given ambiguous context or asked to verify correctness without a real execution environment, it hallucinates confidently. The engineer must remain the verification oracle — AI cannot substitute for running the actual simulation.